| CONCEPT | DIMENSION | CODING |
|---|---|---|
| **Recursion type**<br>What kind of recursion is it? | SOURCE (HTDP, MIT 6.102) | **Structural recursion** LYAH ×2 · SICP · HtDP ×7 · DCIC ×2 · MIT 6.102 ×2 · LeetCode ×14<br>Follows the data definition: first/rest, left/right. |
| | | **Generative recursion** LYAH · SICP · HtDP ×2 · CLRS · LeetCode ×15<br>Subproblems are computed, not parts of the data. |
| | CALLS PER STEP (SICP §1.2) | **Linear recursion** LYAH ×2 · SICP · HtDP ×7 · DCIC ×2 · LeetCode ×18<br>One recursive call per step. |
| | | **Tree recursion** LYAH · SICP · HtDP ×2 · CLRS · MIT 6.102 ×2 · LeetCode ×11<br>Two or more recursive calls per step. |
| | PROCESS (SICP §1.2.1) | **Recursive process** LYAH ×3 · SICP ×2 · HtDP ×9 · DCIC ×2 · CLRS · MIT 6.102 ×2 · LeetCode ×28<br>Work deferred and done on the way back. |
| | | **Iterative process** LeetCode<br>State carried down; nothing left to do on return. |
| **Decomposition**<br>How are the smaller instances laid out, and how many calls are drawn? | STRUCTURE | **Nested containment**<br>Calls drawn inside their caller.<br>— *Box-in-box* HtDP<br>— *Brackets* LYAH · LeetCode ×2 |
| | | **Tree (node-link)**<br>Calls linked to their subcalls.<br>— *Call tree* SICP · LeetCode ×3<br>— *State tree* LeetCode ×2<br>— *Choice tree* LeetCode<br>— *Result tree* LeetCode ×6 |
| | | **Linear chain**<br>List of nodes and links.<br>— *Box-and-pointer* HtDP · LeetCode<br>— *Pointer rewiring* LeetCode ×8 |
| | | **Stack** LeetCode ×2<br>Frames piled up by depth. |
| | | **Table** HtDP · LeetCode<br>Args, intermediates and results. |
| | | **Substitution trace** SICP · HtDP ×3 · LeetCode<br>Expression rewritten until a value. |
| | | **Indentation staircase** DCIC · LeetCode<br>Depth shown by indentation. |
| | | **Partition** LeetCode<br>Input split into boxed parts. |
| | | **Instance mapping** LeetCode ×3<br>Size n aligned to size n−1. |
| | | **Metaphor / pictorial** LYAH · HtDP · LeetCode<br>Dolls, mirrors, spirals. |
| | CHOICES | **Choice label** LeetCode ×2<br>The decision named on the edge. |
| | | **Prune / fail + legend** LeetCode ×3<br>Dead branches, keyed in a legend. |
| | ELISION (CALLS) | **Full expansion** SICP · LeetCode ×2<br>Small input, every call drawn. |
| | | **Patterned elision** CLRS<br>Rule implied by a few levels. |
| | | **Unpatterned elision** LeetCode<br>No inference can be made. |
| | | **Deferred call**<br>Subcall left unexpanded.<br>— *Trace term* HtDP<br>— *Done box* LeetCode ×2 |
| | | **Trusted subcall** LeetCode ×5<br>The subcall's result is used without drawing inside it (leap of faith). |

| CONCEPT | DIMENSION | CODING |
|---|---|---|
| **Self-referential data**<br>Which data refers to itself, and what sits inside one node? | DATA | **List** HtDP · DCIC<br>A list whose rest is again a list. |
| | | **Tree** MIT 6.102 ×2<br>A tree whose children are trees. |
| | | **Nested doll** HtDP<br>A pictorial object holding a smaller copy. |
| | NODE CONTENT | **Concrete data** DCIC · MIT 6.102<br>Values of the input itself. |
| | | **Call with arguments** SICP · LeetCode ×3<br>The call as written, concrete args. |
| | | **State tuple** LeetCode ×3<br>Only the changing parameters. |
| | | **Partial expression** SICP · HtDP · LeetCode<br>Half-evaluated term, as in a trace. |
| **Return & combine**<br>How do results come back, and how many returns are drawn? | FLOW | **Upward only** HtDP<br>Results merging toward the root. |
| | | **Round trip** SICP · LeetCode ×7<br>Down to the base case, then back up. |
| | | **Rewrite in place** SICP · HtDP ×2<br>Time runs down the lines. |
| | MARKS | **Return value** LeetCode ×3<br>Value written on the way back. |
| | | **Combine step** SICP · HtDP · LeetCode ×2<br>How sub-results merge is drawn. |
| | | **Traversal path** SICP<br>Evaluation order traced. |
| | | **Call-step label** LeetCode ×2<br>Each frame or step numbered. |
| | ELISION (RETURNS) | **All returns drawn** SICP ×2 · HtDP ×3 · LeetCode ×7<br>Every call's result is shown coming back. |
| | | **Some returns drawn** LeetCode<br>Only some results, e.g. the final one. |
| | | **Returns not drawn** LeetCode ×4<br>Calls only; for a recursive process the way back is missing. |
| **Base case**<br>Is the stopping point shown, and how? | MARKING | **Marked** LeetCode ×4<br>Labelled or highlighted as the base case. |
| | | **Reached, not marked** LYAH<br>The last call is drawn but not singled out. |
| | | **Dead-end leaf** LeetCode<br>A failed branch ends the recursion. |
| **Code Correspondence**<br>How does it tie back to the code? | LINK | **Diagram is code** SICP · HtDP · DCIC<br>Trace written in the program's own syntax. |
| | | **Linked to the code**<br>Placed next to it, arrowed into it, or sharing its names.<br>— *Side by side* LYAH ×2 · SICP · LeetCode ×4<br>— *Arrows into code* HtDP · LeetCode<br>— *Shared identifiers* SICP · LeetCode ×3 |
| | | **Linked through math or prose**<br>An equation or a written explanation, not the code.<br>— *Recurrence only* CLRS · LeetCode ×2<br>— *Prose only* LeetCode ×2 |

LYAH Learn You a Haskell · SICP Structure and Interpretation of Computer Programs · HtDP How to Design Programs · DCIC A Data-Centric Introduction to Computing · CLRS Introduction to Algorithms
MIT 6.102 MIT 6.102 course notes · LeetCode LeetCode solutions, vote ≥ 5 · ×n number of examples

**Figure X: A design space for recursion diagrams, read as one table in two halves. Each recursion concept lists the dimensions along which it is drawn.**